A ADDITIONAL EXPERIMENTAL RESULTS

A.1 Parameter Tuning

Query parameters. All queries retrieve $k = 10$ results and are executed single-threaded. For each approximate method, we vary its query-time search parameter to trace the recall–QPS tradeoff: L_{search} for Filtered-DiskANN and UNG, `efSearch` for NaviX and ACORN, *ef* for RWalks and DYLAN, L for UniFilter, and `search_ef` for Curator. Prefilter evaluates the label predicate exactly and performs an exact L_2 scan over the survivors, and therefore appears as a single point in the recall–QPS plots.

Construction parameters. We tune the build-time parameters of every method, DYLAN included, on every dataset under one common protocol. Each configuration is scored by its QPS at recall 0.9 — the operating point used throughout Sec. 4 for most experiments — on containment and on overlap queries, and we select the configuration maximizing the geometric mean of the two; if only one query type reaches this recall, the configuration is scored on that type alone, and if neither does, configurations are ranked by the geometric mean of their highest attained recalls. The swept grids, with the remaining parameters fixed to the values recommended by the corresponding papers, are: $(L_{\text{build}}, R) \in \{(96, 64), (144, 96), (192, 128), (240, 160)\}$ with $\alpha = 1.2$ for Filtered-DiskANN; $(R, L) \in \{(16, 100), (32, 100), (32, 200), (48, 200)\}$ with $\alpha = 1.2$, 6 cross-group edges and 16 query entry vectors for UNG; $(\texttt{nlist}, B_{\max}) \in \{(16, 64), (32, 128), (64, 256), (64, 512)\}$ with beam size 4 for Curator; and $M \in \{16, 32, 48, 64\}$ with $ef_c = 400$ for NaviX, RWalks, ACORN-1 ($\gamma = 1$, $M_\beta = 2M$), ACORN-γ ($\gamma = 100$, $M_\beta = 64$), UniFilter ($M_p = n$), and DYLAN, ACORN-1 and ACORN-γ being swept independently. We further extend Filtered-DiskANN's `detect_common_filters` to support both containment and overlap predicates. Table 6 reports the selected values. Each baseline is thus evaluated under its own best configuration rather than a default one, and DYLAN receives no larger construction budget: except on YTB-Video, its selected M is 16, no larger than that of any baseline. RWalks is omitted on YFCC, as its memory consumption exceeds the available 1 TB.

A.2 Impact of Query Selectivity

Fig. 28 shows the additional results of the impact of filter selectivity on query performance on TripClick and YFCC omitted in Sec. 4. Recall that it is infeasible to generate containment queries spanning a continuous range of target selectivity due to the label distributions in YFCC and TripClick. We only report overlap queries on these two datasets. These results are consistent with the trends presented in Sec. 4. It can be observed that our method generally outperforms the other approaches, except on YFCC under very low selectivity.

A.3 Query Performance of Complex Predicate

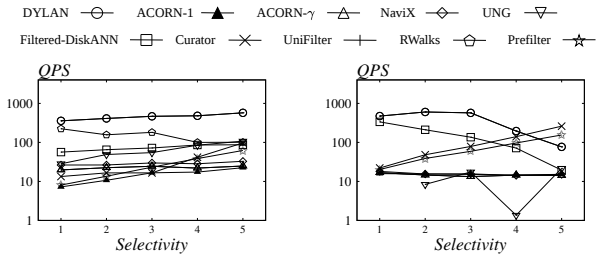
Fig. 32 shows the results on the remaining 4 datasets omitted in Sec. 4. Our method achieves performance comparable to Curator on YFCC and YTB-Video, significantly outperforming other approaches. On the YTB-Audio dataset, our method outperforms all other methods, including Curator, by two orders of magnitude. This demonstrates that our method maintains superior performance across all datasets and under any query conditions.

Table 4: DYLAN memory cost breakdown by component (GB)

Dataset	Vector	Labels	Trie	Graph	Other	Total
TripClick	3.02	0.06	0.01	0.41	0.06	3.56
LAION1M	1.91	0.05	0.01	0.48	0.06	2.52
YFCC	0.72	0.07	1.09	0.77	0.15	2.80
YTB-Audio	2.39	0.27	0.28	3.08	0.38	6.38
YTB-Video	3.82	0.05	0.09	2.12	0.08	6.15
Yelp1M	1.43	0.06	0.03	0.71	0.08	2.32

Table 5: Absolute Pearson correlation $|r|$ between label-set Jaccard similarity and L_2 distance. Object pairs are grouped by $\max(|f_i|, |f_j|)$.

Dataset	$[1, 2]$	$[3, 5]$	$[6, 10]$	> 10	Overall
TripClick	0.0055	0.0078	0.0528	–	0.0055
LAION1M	0.0408	0.0617	–	–	0.0428
YFCC	0.1891	0.0028	0.0073	0.0012	0.0217
YTB-Audio	0.0018	0.0011	0.0028	0.0031	0.0009
YTB-Video	0.0216	0.0209	0.0217	0.0047	0.0206
Yelp1M	0.5742	0.5089	0.4711	0.4458	0.4791



(a) TripClick (b) YFCC

Figure 28: Impact of selectivity on different datasets.

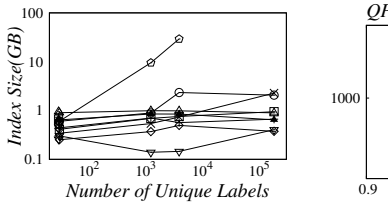


Figure 29: Index size vs. $|\mathcal{L}|$.

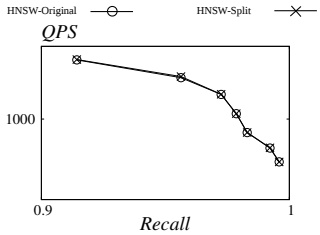
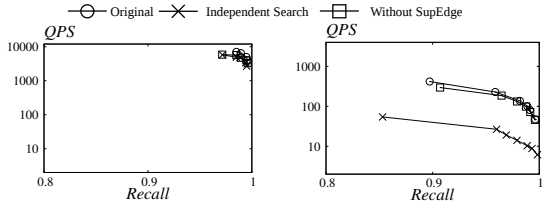


Figure 30: Memory layout.

A.4 Memory Breakdown of DYLAN Index

Tab. 4 reports the memory consumption of DYLAN by component. We separately account for the global vector array, object labels, the label trie, materialized graph indices, and other auxiliary structures. The "Other" category includes the *belong* arrays and the label inverted lists. Since each vector is stored only once in the global vector array, materializing an object in multiple graph indices only introduces graph metadata rather than additional vector copies. The breakdown also reflects the different characteristics of the datasets. For example, YFCC incurs a relatively large trie cost due to its large label universe and complex label sets, whereas the graph indices constitute the main additional cost on YTB-Audio. Overall, the trie and auxiliary structures remain moderate compared with the vector and graph storage, confirming that the additional structures introduced by DYLAN do not dominate its memory consumption.



(a) Containment (b) Overlap

Figure 31: Ablation study of optimizations: YFCC.

A.5 Label Scalability

Sec. 4 evaluates scalability along two dimensions, the number of vectors and the size of the label universe, and reports only the former; we report the latter here. Fig. 29 plots the index size of each method against the number of distinct labels over the five 1M-scale datasets (all datasets except YTB-Audio), whose label universes span from 29 labels on TripClick to 181,931 on YFCC. Since all five datasets hold roughly the same number of vectors, the raw vector payload is essentially constant across them and would mask the differences of interest; we therefore exclude it for every method and report only the index structures built on top of it. The index size of DYLAN grows by only 4.4× across the entire span and follows a trend similar to most existing methods, indicating that neither the label trie nor the auxiliary structures grows substantially with the label universe. RWalks is the clear exception: it encodes labels into vector-like representations, so its index size increases sharply as labels accumulate, which is also why it exceeds the available 1 TB on YFCC. These results are consistent with the Label Scalability column of Tab. 1 and confirm that DYLAN scales effectively to large label universes.

A.6 Impact of Memory Layout

DYLAN stores vectors and adjacency lists separately, so reaching a neighbor's vector requires an extra indirection through its object ID. To isolate the cost of this indirection, we compare two layouts of a standard HNSW graph on LAION1M in Figure 30. The first follows the original implementation of HNSWlib [2], where each vertex stores its vector contiguously with its adjacency list, avoiding an additional pointer dereference to locate the vector. The second is the separated layout used by DYLAN, where adjacency lists hold only object IDs and all vectors reside in a global array, so expanding a neighbor requires a second random access into that array. The two layouts achieve nearly identical query performance, differing by less than 1%: the indirection is amortized over the distance computation that follows it, and the vector array is accessed in the same order regardless of layout. In DYLAN the effect is even smaller, since many visited neighbors fail the label filter and are skipped without any vector access at all. Together with Fig. 30, this confirms that the separated layout, which lets an object appear in multiple materialized graphs without duplicating its vector, does not trade memory for query latency.

A.7 Label-Vector Correlation by Label-Set Size

We analyze whether label-set similarity is correlated with vector proximity, and whether such correlation depends on how many labels the objects carry. For each dataset, we uniformly sample 1M distinct object pairs and compute the Pearson correlation r between their label-set Jaccard similarity and their L_2 distance, reporting $|r|$ since negating the distance only flips the sign. A single overall correlation can be misleading here, as pairs with few labels and pairs with many labels differ in both their Jaccard values and their distance distributions: the overall coefficient absorbs these cross-bucket shifts under a global normalization. We therefore also group the sampled pairs by $\max(|f_i|, |f_j|)$ into four ranges—$[1, 2]$, $[3, 5]$, $[6, 10]$, and > 10—and report the within-bucket correlation for each.

Tab. 5 reports the results. On the five benchmark datasets, the overall correlations are consistently very small, the largest being $|r| = 0.0428$ on LAION1M, and the bucket-level correlations stay below 0.0617 in almost all cases. The only exception is the $[1, 2]$ bucket of YFCC with $|r| = 0.1891$, which draws on just 3,774 sampled pairs and therefore has limited support. Grouping by label-set size thus does not change the picture: on these datasets, objects with similar label sets are not systematically closer in the vector space, at any label-set size. Yelp1M is the exception by construction: its labels are business categories attached to review text, so an overall $|r|$ of 0.4791 together with consistently high bucket-level correlations makes it a complementary workload on which label similarity does carry information about vector proximity.

A.8 Impact of Update

Sec. 4.2 reports both update workloads on YTB-Video only. Here we give the results on the remaining datasets: Figs. 33 and 34 for the insertion-only workload, Figs. 35 and 36 for the mixed workload, in each case for containment and overlap queries respectively, and Fig. 37 for the update time under the mixed workload. The protocol follows Sec. 4.2: an index built directly on the final object set is compared against one that reaches the same set through updates. Across all datasets and both workloads, the two recall-QPS curves stay close, confirming that the threshold-based maintenance scheme preserves index quality under insertions, and that lazy deletions leave the graph topology untouched.

A.9 Ablation study of optimizations

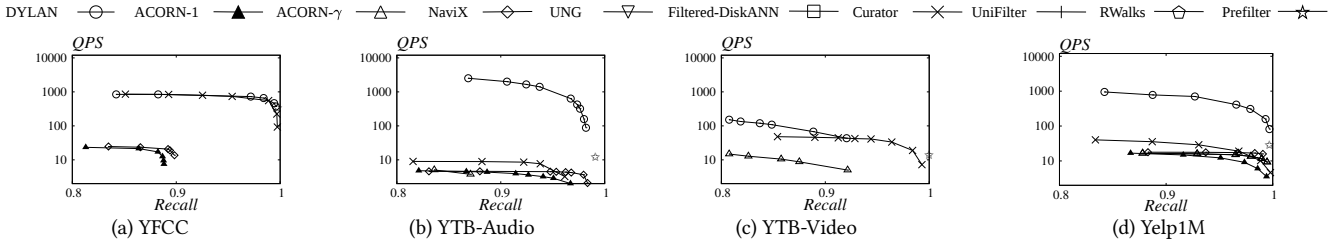
Sec. 4.2 evaluates the two optimizations of DYLAN on YTB-Audio only; Fig. 31 reports the same ablation on YFCC. Replacing multi-graph search with independent search again degrades the QPS-recall tradeoff, most visibly on overlap queries, where a filter spans more trie regions and the key-ancestor graphs therefore matter most for cross-region connectivity. Disabling SUPPLEMENTEDGES, in contrast, has almost no effect on YFCC: its materialized graphs are already well connected, so the supplemented edges are rarely traversed. This confirms the dataset-dependence noted in Sec. 4.2—the multi-graph search is the essential component, while SUPPLEMENTEDGES helps only where per-region connectivity is insufficient.

A.10 Impact of Parameter ef_c on DYLAN

Fig. 38 shows the impact of ef_c. As we can observe, increasing ef_c leads to higher indexing time but strengthens the quality of the graph, resulting in better performance. This improvement in connectivity enhances query performance. Based on the experimental results, we set $ef_c = 400$ as the default parameter as it gains a nice trade-off between the query performance and indexing cost.

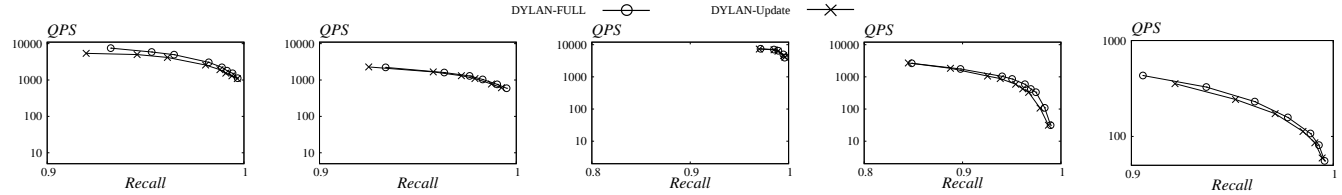
Table 6: Selected construction parameters for each method and dataset. Here R and M denote the graph out-degree, L_{build} and L denote the construction candidate size, and `nlist` and $B_{\max}$ denote Curator's branching factor and maximum leaf size. ACORN-1 and ACORN-γ are tuned independently, and "–" indicates that the method is omitted due to out-of-memory.

Dataset	Filtered-DiskANN	UNG	NaviX	ACORN-1	ACORN-γ	Curator	RWalks	UniFilter	DYLAN
	(L_{build}, R)	(R, L)	M	M	M	(`nlist`, $B_{\max}$)	M	M	M
LAION1M	(144, 96)	(48, 200)	48	48	48	(64, 512)	48	16	16
TripClick	(192, 128)	(32, 200)	64	48	48	(64, 512)	32	16	16
YFCC	(240, 160)	(16, 100)	48	48	48	(64, 512)	–	32	16
YTB-Audio	(240, 160)	(48, 200)	48	64	64	(64, 512)	48	48	16
YTB-Video	(240, 160)	(48, 200)	64	64	64	(64, 512)	64	16	64
Yelp1M	(240, 160)	(16, 100)	48	64	64	(64, 512)	48	64	16



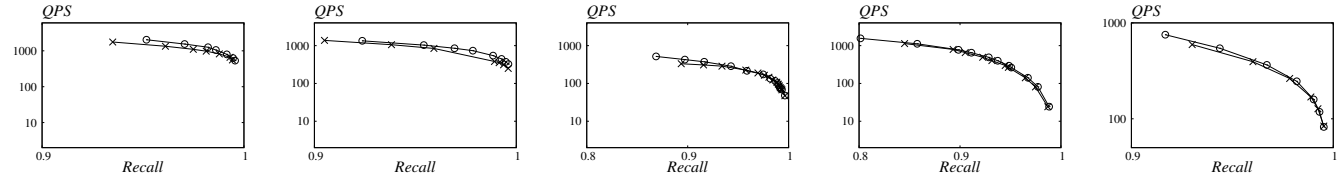
(a) YFCC (b) YTB-Audio (c) YTB-Video (d) Yelp1M

Figure 32: Complex predicate.



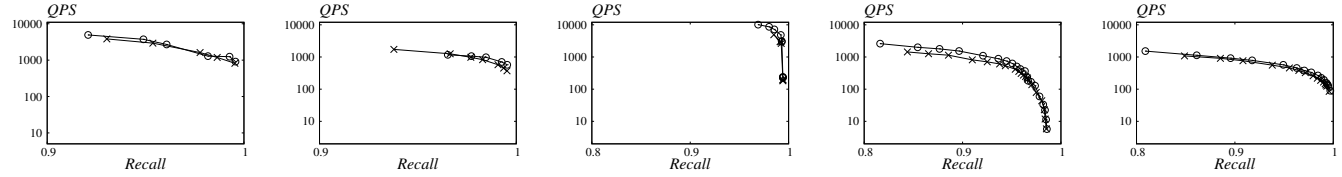
(a) LAION1M (b) TripClick (c) YFCC (d) YTB-Audio (e) Yelp1M

Figure 33: Impact of insertion-only update: containment queries.



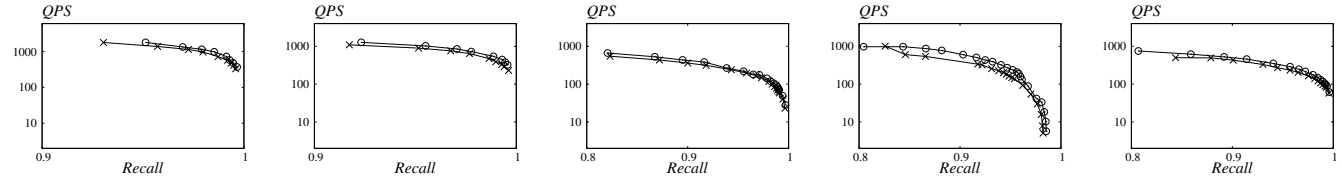
(a) LAION1M (b) TripClick (c) YFCC (d) YTB-Audio (e) Yelp1M

Figure 34: Impact of insertion-only update: overlap queries.



(a) LAION1M (b) TripClick (c) YFCC (d) YTB-Audio (e) Yelp1M

Figure 35: Impact of mixed updates: containment queries.



(a) LAION1M (b) TripClick (c) YFCC (d) YTB-Audio (e) Yelp1M

Figure 36: Impact of mixed updates: overlap queries.

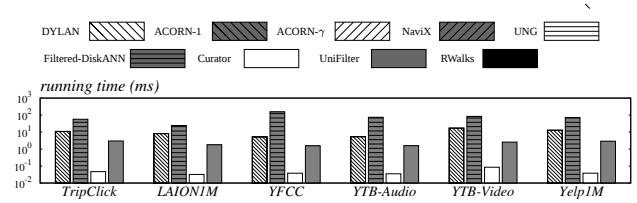
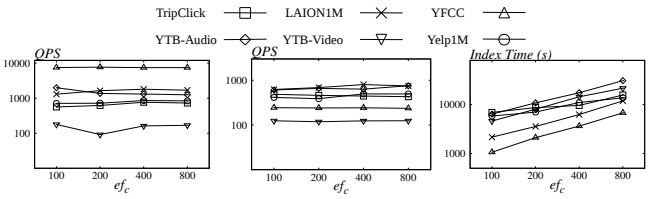


Figure 37: Mixed update time.



(a) containment (b) overlap (c) indexing cost

Figure 38: Impact of ef_c to query and indexing cost.

B PROOFS

Proof of Lemma 3.4. Let $v = u.\text{index}$. By construction, v is an ancestor of u, so $O_u \subseteq O_v$, and u shares $v.G$ only when $\text{log_size}(u) = \text{log_size}(v)$. Hence, for some k,

$$2^k \le |O_u| \le |O_v| < 2^{k+1},$$

which implies $|O_v| < 2|O_u|$. Therefore, $v.G$ is a 2-approximate index for O_u by Definition 3.3.

Proof of Lemma 3.5. We prove by contradiction. Assume that node u has two distinct children c_1 and c_2 such that $\log_2(|O_{c_1}|) = \log_2(|O_u|)$ and $\log_2(|O_{c_2}|) = \log_2(|O_u|)$. Without loss of generality, let $|O_{c_1}| \le |O_{c_2}|$. Since O_u is the set of all objects in the subtree of u and c_1, c_2 are disjoint subtrees, we have $|O_u| \ge |O_{c_1}| + |O_{c_2}| \ge 2|O_{c_1}|$. Consequently, $\log_2(|O_u|) \ge \log_2(2|O_{c_1}|) = \log_2(|O_{c_1}|) + 1 = \log_2(|O_u|) + 1$, which is a contradiction. Therefore, the assumption is false, and the lemma holds.

Proof of Lemma 3.8. For the object o, it only belongs to the graph indexes of its ancestors in the Trie; hence there are at most $O(|f_o|)$ such nodes. Moreover, because a node u maintains its own index ($u.index = u$) only when $\text{log_size}(u) \ne \text{log_size}(u.p)$, the number of distinct log_size values along the root-to-leaf path is $O(\log n)$. Therefore, o can belong to at most $O(\log n)$ such indexes. Combining the two bounds yields the result.

Proof of Theorem 3.9. The space complexity of our index consists of the following components: the Trie structure, the inverted lists I, the belong array, and all the HNSW graph indexes. Each object o is inserted into the Trie by traversing (or creating) at most $|f_o|$ nodes. Hence the Trie and the associated inverted lists require $O\left(\sum_{o\in O} |f_o|\right)$ space. The $belong_{o,i}$ indicates whether object o belongs to a graph of logarithmic size i. Since i can take at most $O(\log n)$ distinct values, the total size of the belong array is $O(n \log n)$. Through Lemma 3.8, the overall space occupied by all graph indexes is $O\left(\sum_{o\in O} \min\{|f_o|, \log n\}\, M\right)$ Summing the three parts, the total space complexity is

$$O\left(n \log n + \sum_{o\in O} |f_o| + \sum_{o\in O} \min\{|f_o|, \log n\}\, M\right).$$

For construction time, we first build the Trie and the inverted lists, which takes $O\left(\sum_{o\in O} |f_o|\right)$ time. Initializing and updating the belong array costs $O(n \log n)$. The dominant cost is the HNSW insertion: each object o is inserted into at most $\min\{|f_o|, \log n\}$ graphs, and each insertion has complexity c_{upd} (the time to insert one point into a plain HNSW graph). For lines 5–8 of Algo. 1, each element o is subject to the `supplementEdges` operation at most $\min\{l_{\text{th}}, |f_o|\}$ times, where the cost of each such operation is c_q, which denotes the cost of performing one query on DYLAN. Hence the total construction time is

$$O\left(n \log n + \sum_{o\in O} |f_o| + \sum_{o\in O} \min\{|f_o|, \log n\}\, (c_{\text{upd}} + c_q)\right).$$

Proof of Theorem 3.12. For containment queries, we access ancestors of nodes in $I_{\max}$ and examine the prefix of each node. For overlap queries, we access ancestors of nodes from the inverted list of every label in f_q and also examine the prefix of each node. The number of ancestors of a node u is $|\text{prefix}(u)|$, and each ancestor can be accessed in $O(1)$ time. Now consider a complex query. For any node $u \in I_l$ where $l \in \mathcal{L}_\Phi$, if u is inspected during a merge step in ComplexSearch, the associated cost is $\text{prefix}(u)$. In the worst-case scenario, every merge operation in ComplexSearch includes u. Since a complex predicate comprising $|\Phi|$ containment or overlap queries entails $|\Phi|$ merge operations, the overall complexity, when combined with the costs of ContainmentSearch and OverlapSearch, is given by $O\left(\sum_{l\in\mathcal{L}_\Phi} \sum_{u\in I_l} |\text{prefix}(u)| \cdot |\Phi|\right)$. Therefore, the theorem holds.

Proof of Theorem 3.15. Operations on the Trie and updates to the inverted lists take $O(|f_o|)$ time. Traversing the path from the root to the leaf also costs $O(|f_o|)$. For HNSW insertion and supplementEdge, an object belongs to at most $O(\min\{|f_o|, \log n\})$ graphs, so the total insertion cost is $O(\min\{|f_o|, \log n\} \cdot (c_{\text{upd}} + c_q))$. Adjusting the index assigned to a node may involve copying or deleting an index; one such operation on node u costs $O(|O_u|M)$ and is triggered only when a change in $|O_u|$ alters $\text{log_size}(u)$. After one such change, the next copy/delete on this index occurs after $\Omega(|O_u|)$ insertions into O_u. Hence the amortized cost per insertion contributed by copying/deleting is $O(M)$. Since an insertion affects at most $|f_o|$ subtrees, the total amortized overhead from copying/deleting is $O(|f_o|M)$.

Rebuilding the index for node u costs $O(|O_u|c_{\text{upd}})$. After a rebuild, $u.G$ contains at most $2^{\text{log_size}(u)+1} - 1$ nodes, and the next rebuild is triggered after $\Omega(|O_u|)$ further insertions. Therefore, the amortized cost per insertion from rebuilding is $O(c_{\text{upd}})$. Because an insertion touches at most $\min\{|f_o|, \log n\}$ graphs, the total amortized rebuilding overhead is $O(\min\{|f_o|, \log n\} \cdot c_{\text{upd}})$. Summing all parts yields an amortized time complexity per update of $O(|f_o|M + \min\{|f_o|, \log n\} \cdot (c_{\text{upd}} + c_q))$. Moreover, the graph $u.\text{index}.G$ is allowed to contain at most $2^{\text{log_size}(u)+2} - 1$ vertices, so

$$\frac{|O_{u.\text{index}.G}|}{|O_u|} \le \frac{2^{\text{log_size}(u)+2} - 1}{2^{\text{log_size}(u)}} < 4,$$

assuring that the index remains a 4-approximate index for O_u.